

Progetto di Ingegneria della Conoscenza A.A. 2025/2026

SMART MAINTENANCE HUB

Un'architettura neuro-simbolica per il monitoraggio e la logistica di
manutenzione

Gruppo di Lavoro:

Mattia Cacciapaglia

Mail:

m.cacciapaglia13@studenti.uniba.it

Repository GitHub:

<https://github.com/mattiacaccia/Smart-Maintenance-Hub>

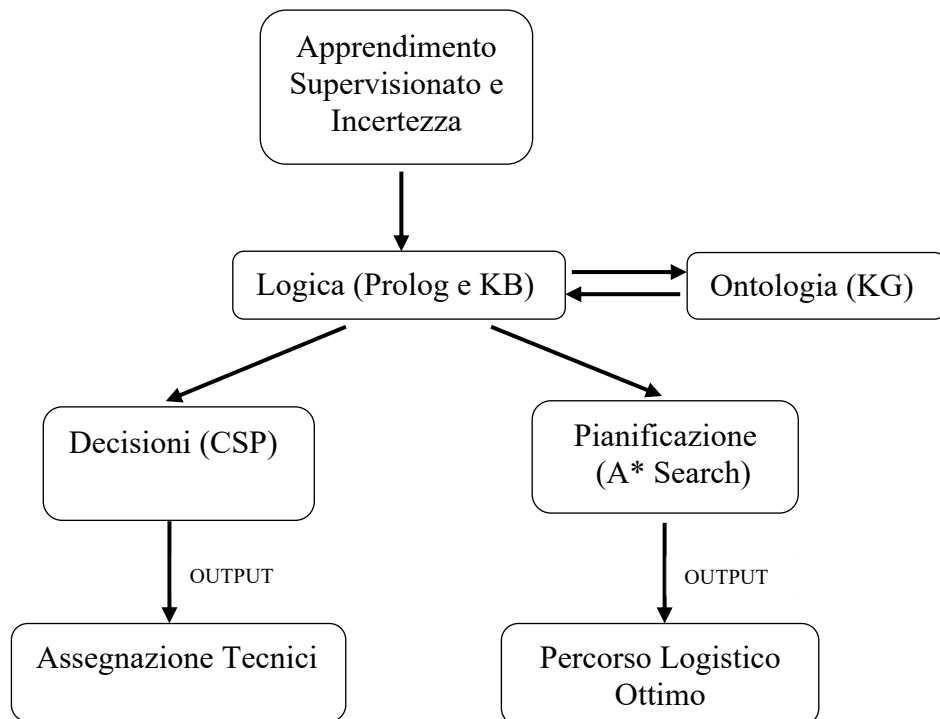
Sommario

Introduzione.....	3
Argomenti di interesse	4
Apprendimento Supervisionato e Incertezza.....	4
Sommario	4
Preprocessing dei Dati	5
Strumenti Utilizzati.....	7
Decisioni di Progetto	7
1.1 Random Forest (Regressione)	7
1.2 Logistic Regression (Classificazione)	7
1.3 Validazione Statistica (Cross-Validation)	8
2. Rete Bayesiana (Gestione dell'Incertezza)	8
Valutazione	9
Random Forest	9
Regressione Logistica	12
Rete Bayesiana	14
Ontologie e Ragionamento	16
Sommario	16
Strumenti utilizzati	16
Decisioni di progetto	16
Logiche Descrittive	16
Logiche proposizionale/Clausole di Horn	18
Integrazione (Modulo di Ragionamento)	19
Valutazione	19
Ricerca di Soluzioni e Ragionamento con Vincoli.....	20
Sommario	20
Strumenti utilizzati	20
Decisioni di progetto	20
CSP (Constraint Satisfaction Problem).....	20
Ricerca di soluzioni in spazi di stati	21
Valutazione	22
Conclusioni	23
Esito	23
Sviluppi Futuri.....	24
Riferimenti bibliografici.....	24

Introduzione

Il progetto Smart Maintenance Hub affronta il dominio della **manutenzione predittiva industriale**, dove la capacità di anticipare un guasto non è sufficiente se non accompagnata da una strategia operativa di intervento efficace.

Si è desiderato integrare gli strumenti discussi durante il corso per realizzare uno strumento “intelligente” e dinamico che permetta di trasformare il monitoraggio sensoriale in una strategia decisionale autonoma, capace di colmare il divario tra il dato numerico grezzo (segnale sensoriale) e l'azione risolutiva (pianificazione della riparazione) ottimizzando l'impiego del personale tecnico.



Argomenti di interesse

1. Apprendimento Supervisionato e Incertezza (Cap. 7, 9, 10)

- a. Apprendimento Supervisionato
- b. Validazione e Sovradattamento
- c. Ragionamento e Incertezza (*Belief Network*)
- d. Apprendimento Probabilistico

2. Rappresentazione della Conoscenza e Ragionamento (Cap. 5, 15, 16)

- a. Clausole Definite e Inferenza
- b. Rappresentazione e Ragionamento Relazionale
- c. Knowledge Graph e Ontologie

3. Ricerca di Soluzioni e Ragionamento con Vincoli (Cap. 3, 4)

- a. Ricerca Informata (Euristica)
- b. Constraint Satisfaction Problems (CSP)
- c. Risoluzione di CSP tramite Ricerca
- d. Risoluzione di CSP con vincoli soft e hard
- e. Ottimizzazione

Apprendimento Supervisionato e Incertezza

Sommario

In questa sezione viene descritta l'analisi e l'elaborazione dei dati (preprocessing dei dati) e come vengono utilizzati dai modelli di **regressione ensemble**, per stimare la vita residua (RUL) dei motori turbofan basandosi su flussi di dati sensoriali, e di **classificazione**. Poiché i sensori in ambito industriale sono soggetti a rumore e guasti parziali, i risultati del modello vengono filtrati da una **Rete Bayesiana** che valuta l'incertezza e la coerenza tra la predizione e il contesto operativo.

Il dataset utilizzato è reperibile su Internet sulla piattaforma [Kaggle](#) al seguente link:
[NASA Turbofan Jet Engine DataSet](#)

Preprocessing dei Dati

Per prima cosa si è analizzato il dataset attraverso i dati presentati e la documentazione associata di quest'ultimo

Per poter allenare il modello abbiamo preso in considerazione il file txt di train 'train_FD001.txt', il file txt di test 'test_FD001.txt' e il file txt 'RUL_FD001.txt' che contiene solamente il valore della RUL reale per ogni motore presente nel file test_FD001.txt' utile per eseguire la valutazione del modello

Il dataset di train 'train_FD001.txt' è presentato come una matrice di numeri di 26 colonne, separate da spazi, suddivise nel seguente modo:

- 1) unit number (numero del motore analizzato) (unit)
- 2) tempo, in cicli (cycle)
- 3) prima configurazione operativa (os1)
- 4) seconda configurazione operativa (os2)
- 5) terza configurazione operativa (os3)
- 6 -26) misurazione del sensore i , $1 \leq i \leq 21$. (si)

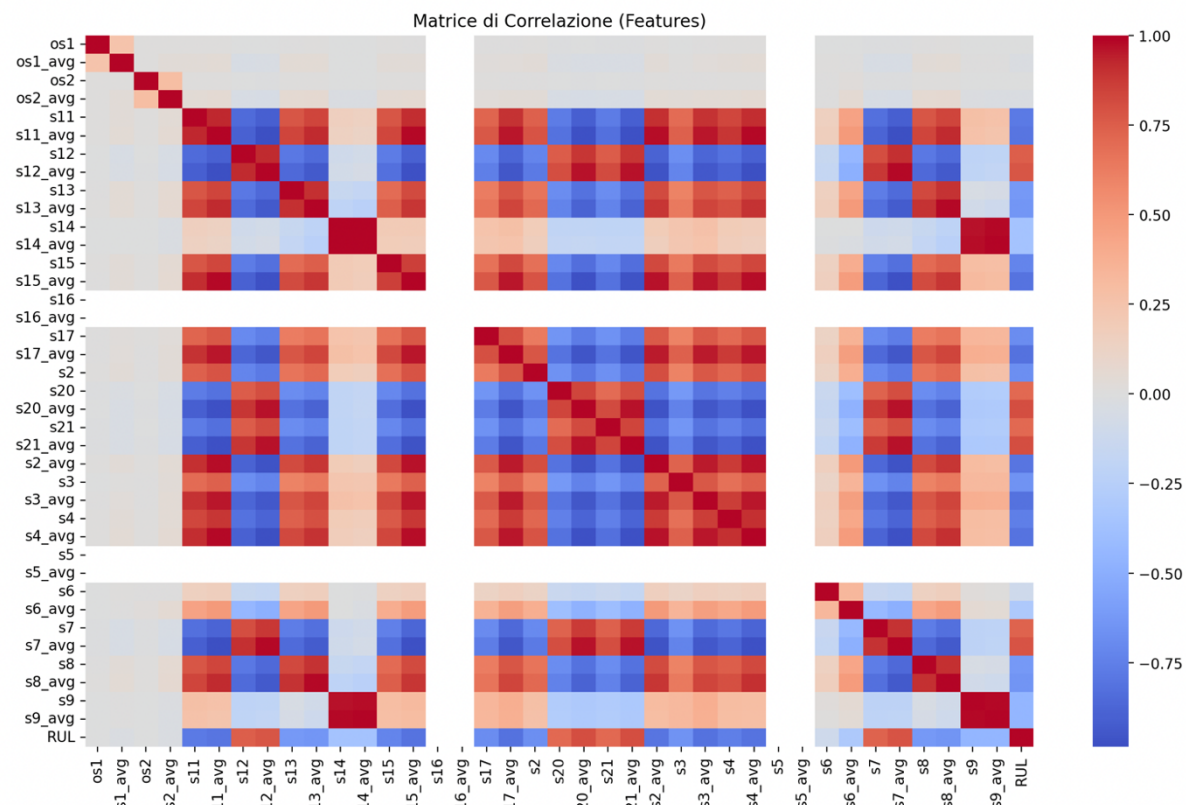
Quindi ogni riga rappresenta un'istantanea dei dati acquisita durante un singolo ciclo operativo

Una volta analizzato il dataset si sono trasformati i dati grezzi dei sensori NASA C-MAPSS del dataset in conoscenza strutturata:

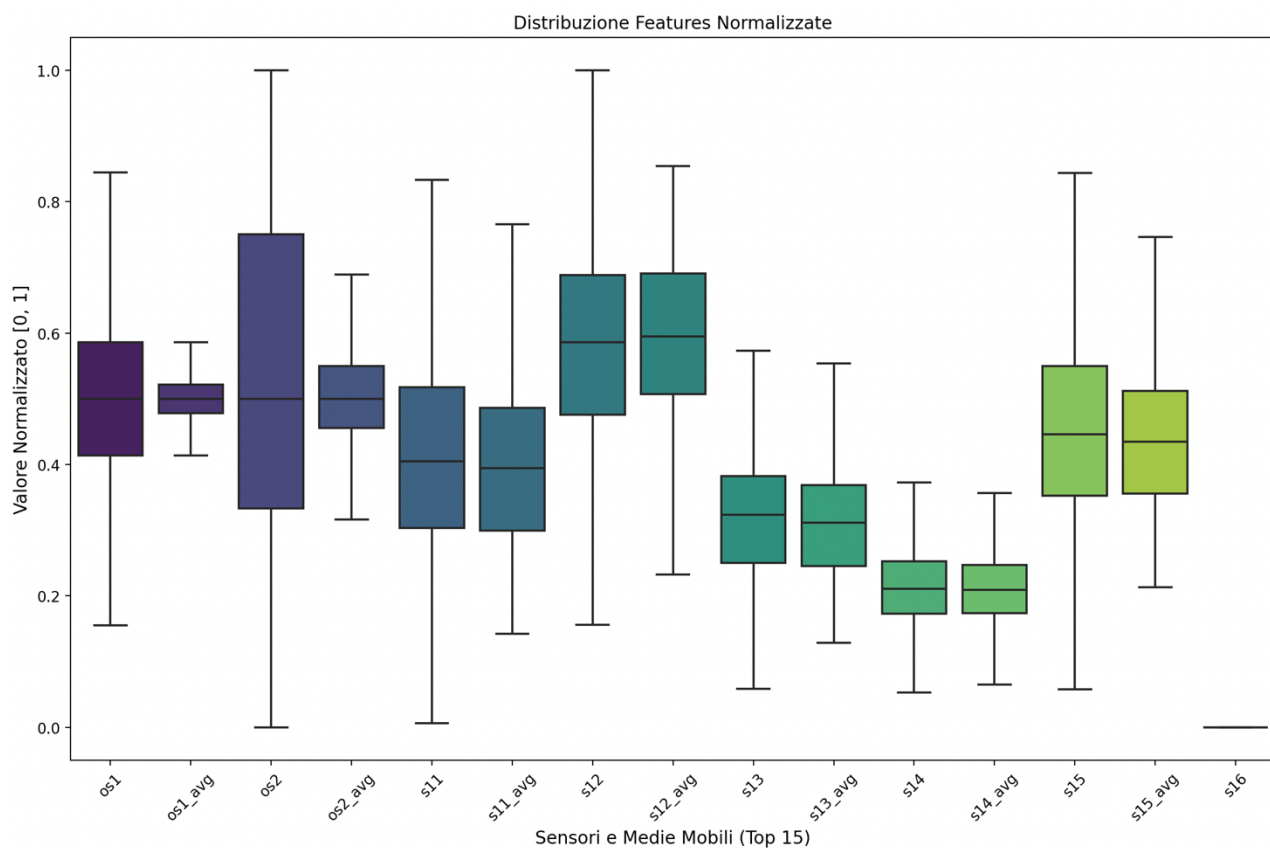
- **Aggiunta della RUL e nominazione delle colonne:** il valore della RUL non ci è dato esplicitamente pertanto è stato calcolato e inserito come ulteriore colonna all'interno della matrice del dataset e per comprendere la matrice numerica ogni colonna è stata nominata come descritto precedentemente (unit, cycle, os1, os2, os3, s1, ..., s21)
- **Pulizia e Selezione:** Identificazione e rimozione dei sensori con varianza nulla (costanti), che non apportano valore informativo.
- **Normalizzazione (Scaling):** Applicazione del MinMaxScaler per portare tutti i segnali nel range [0, 1] per eliminare il **bias di scala**: garantisce che sensori con unità di misura differenti contribuiscano equamente alla funzione di costo del modello di regressione.
- **Feature Engineering (Smoothing):** Utilizzo di una **Media Mobile (window=15)** per stabilizzare i segnali e catturare il trend di usura a lungo termine (dato smoothed indicato con si_avg) oltre ai dati grezzi dei sensori (si) che catturano anomalie istantanee.
- **Feature Importance:** Attraverso l'analisi del Random Forest, sono stati identificati i sensori critici per la degradazione della RUL.

Di seguito sono stati riportati i grafici:

- **Matrice di correlazione:** utilizzata per individuare i legami lineari tra le feature e la RUL, permettendo di validare la selezione dei sensori più informativi ed inoltre conferma l'avvenuta normalizzazione



- **Distribuzione delle top 15 features normalizzate**



Strumenti Utilizzati

Sono state usate le seguenti librerie:

- **Scikit-learn:** Per l'implementazione del Random Forest e le metriche di validazione.
- **Pandas & Numpy:** Per la manipolazione delle serie temporali e il preprocessing.
- **Pgmpy:** Per la costruzione e l'inferenza probabilistica della Belief Network.
- **Matplotlib/Seaborn:** Per la visualizzazione delle matrici di correlazione e importanza.

Decisioni di Progetto

1.1 Random Forest (Regressione)

Il modello scelto per la stima della **RUL (Remaining Useful Life)** è un **Random Forest Regressor**. Si tratta di un algoritmo di *Ensemble Learning* basato sul *Bagging*, che costruisce molteplici alberi di decisione durante l'addestramento e restituisce la media delle loro previsioni.

- **Configurazione del Modello:** Per garantire un modello robusto ed evitare il fenomeno dell'overfitting, sono stati impostati i seguenti iperparametri di regolarizzazione:
 - **n_estimators=100:** Il numero di alberi nella foresta.
 - **max_depth=10:** Limita la profondità degli alberi per evitare che il modello impari eccessivamente il rumore dei dati.
 - **min_samples_leaf=10:** Assicura che ogni foglia abbia un numero minimo di campioni, stabilizzando la predizione.
 - **max_features='sqrt':** Ogni split dell'albero considera solo una radice quadrata del numero totale di feature, aumentando la diversità tra gli alberi.

```
1.         self.regressor = RandomForestRegressor(  
2.             n_estimators=100,  
3.             max_depth=10,  
4.             min_samples_leaf=10,  
5.             max_features='sqrt',  
6.             random_state=42,  
7.             n_jobs=-1 #esegue in parallelo su tutte le CPU  
8.         )
```

1.2 Logistic Regression (Classificazione)

Mentre la Random Forest fornisce una stima del tempo rimanente, è stato introdotto un secondo modello basato su **Regressione Logistica** per gestire la classificazione binaria dello stato del motore (*Safe vs Alarm*).

- **Configurazione del modello:** Per quanto riguarda gli iperparametri di regolarizzazione, abbiamo settato **max_iter=1000** per garantire la convergenza anche su dataset complessi

```
1. self.classifier = LogisticRegression(max_iter=1000, random_state=42)
```

È stato addestrato mappando la RUL su un target booleano, dove il valore 1 (Allarme) scatta quando la vita residua scende sotto la soglia critica di 30 cicli.

```
1. self.feature_names = feature_names
2. X = df_train[feature_names]
3. y_reg = df_train['RUL']
4. y_class = (y_reg <= self.threshold).astype(int). #threshold = 30
5.
6. self.classifier.fit(X, y_class)
```

Inoltre, la Regressione Logistica fornisce anche una **probabilità (Softmax output)**. Questo valore di confidenza viene passato direttamente al modulo di gestione dell'incertezza (Bayes), permettendo di pesare l'allerta in base al carico operativo del motore.

```
1. y_prob_class = self.classifier.predict_proba(X_test)[: , 1] #probabilità per la belief network
```

1.3 Validazione Statistica (Cross-Validation)

Per questo progetto è stata implementata una **5-Fold Cross-Validation** tramite la funzione `cross_validate`. Il dataset viene diviso in 5 parti (fold); il modello viene addestrato su 4 e testato sulla rimanente, ruotando questo processo 5 volte il che ci aiuta ad ottenere il modello con il miglior set di iperparametri.

2. Rete Bayesiana (Gestione dell'Incerteza)

Il modulo di gestione dell'incerteza ha il compito di convalidare l'allarme generato dai modelli di Machine Learning (Random Forest e Logistic Regression) incrociandolo con il contesto operativo del motore e la conoscenza memorizzata nel Knowledge Graph.

Attraverso il calcolo della probabilità statistica che il motore sia in stato di guasto imminente con Regressione Logistica, la **Rete Bayesiana**, permette al sistema di non ragionare in modo puramente booleano (0/1), ma di **pesare l'incertezza del modello statistico**.

- **Configurazione della belief network:**

La rete è composta da tre nodi principali che interagiscono per calcolare la variabile latente `real_failure` (Vero Guasto):

1. **ML_Output (Nodo di Osservazione):** rappresenta il verdetto della Regressione Logistica. Se il modello ML è incerto (es. probabilità 55%), la rete Bayesiana ridurrà l'importanza di questo segnale a favore dei sensori fisici.
2. **Load (Nodo di Contesto):** estrae dati in tempo reale dai sensori di carico operativo (os). Un carico di lavoro estremo può causare "rumore" nei sensori, portando il ML a dare falsi allarmi. La rete Bayesiana è istruita per "scetticismo" se il carico è fuori norma.
3. **Real_Failure (Nodo Decisionale):** è la variabile che determina se inviare o meno la segnalazione al modulo Prolog. La sua probabilità a priori è influenzata dal **Knowledge Graph**: se il KG indica che il motore appartiene a una flotta con storico di guasti frequenti, la rete diventa più sensibile agli allarmi.


```
1. self.model = DiscreteBayesianNetwork([('load', 'ML_Output'), ('real_failure', 'ML_Output')])
```

Il modulo utilizza l'algoritmo di **Eliminazione di Variabile (Variable Elimination)** per calcolare la probabilità a posteriori del guasto.

- **Caso di Alta Coerenza:** Se il Regressore indica RUL bassa, la Logistica dà alta probabilità di guasto e il Carico è normale → La probabilità di real_failure sale sopra l'85%, attivando l'intervento d'urgenza nel Prolog.
- **Caso di Anomalia dei Sensori:** Se la Logistica dà allarme ma il sensore di carico indica uno sforzo anomalo che "spiega" il comportamento dei sensori → La rete Bayesiana abbassa la probabilità di guasto, evitando un costoso fermo macchina non necessario.

```
1. self.inference = VariableElimination(self.model)
```

Valutazione

La valutazione dei risultati è fondamentale quando si allenano modelli di apprendimento e incertezza

Random Forest

La valutazione della random forest serve a dimostrare quanto il sistema sia preciso nel calcolare i "giorni" (cicli) mancanti alla rottura.

Come già specificato precedentemente, è stata utilizzata una 5-fold cross-validation per garantire la generalizzazione ed ottenere il modello con il miglior set di iperparametri

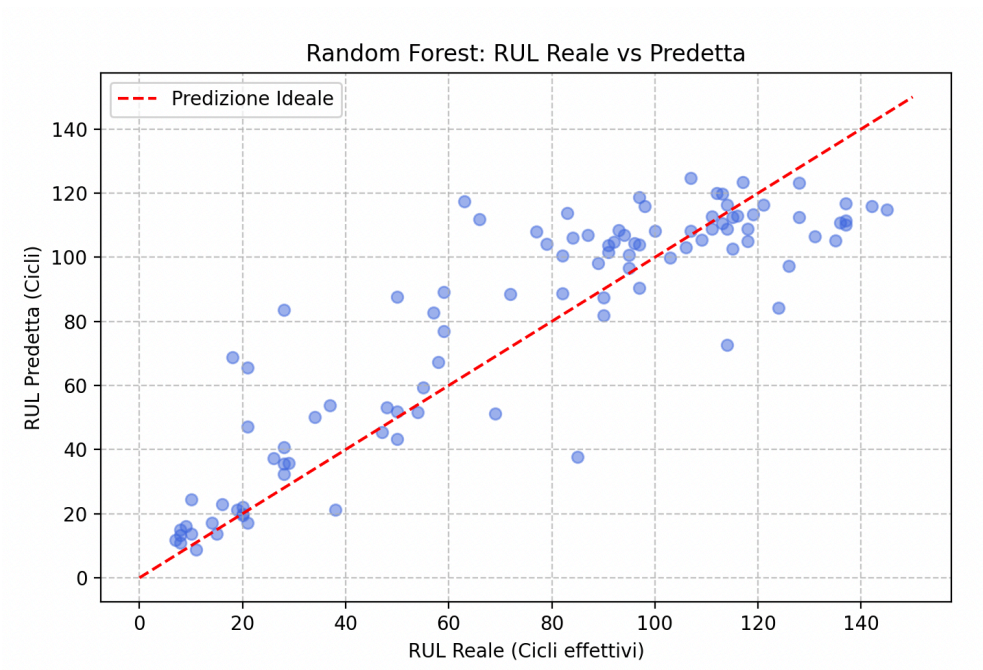
Effettuando dei test sulle metriche di errore si ottengono questi risultati:

MAE (Mean Absolute Error): 14.18

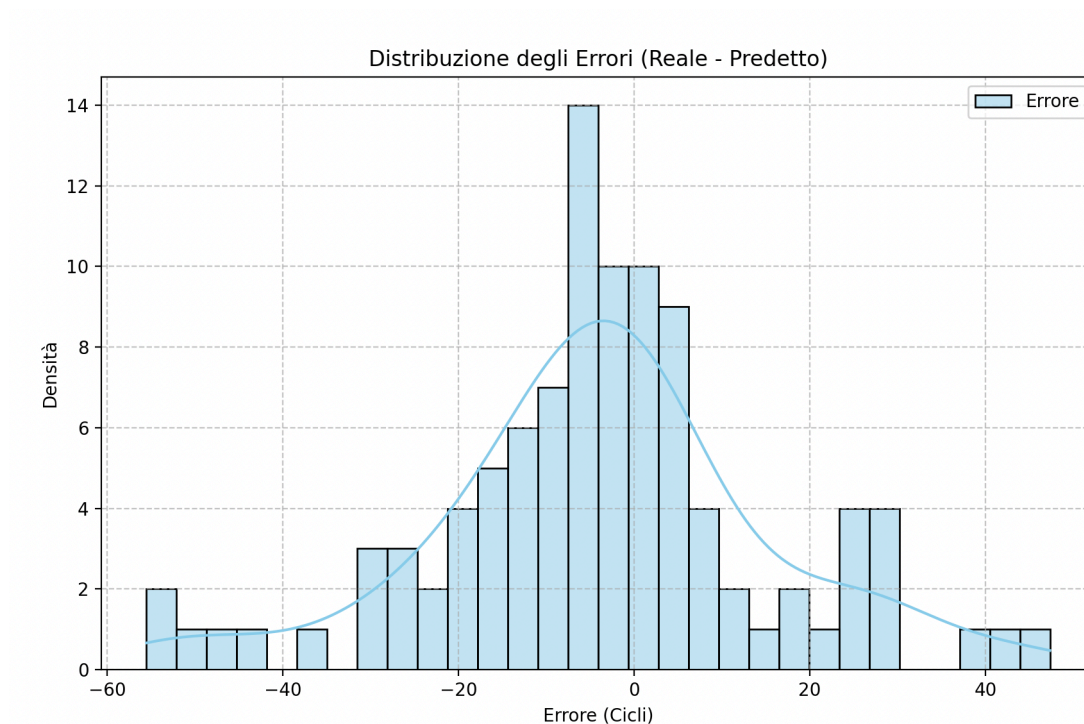
RMSE (Root Mean Squared Error): 18.99

I risultati ottenuti sul test set (100 motori della NASA) sono **estremamente positivi** per un modello basato su Random Forest. In media, il sistema sbaglia la stima della vita residua di circa 14 cicli (MAE) ed in un contesto aeronautico dove la vita totale può superare i 200 cicli, un errore così basso permette una pianificazione della manutenzione con largo anticipo e il fatto che la RMSE sia di poco superiore al MAE indica che esistono alcuni outliers che generano errori più grandi.

Utilizzando la libreria *matplotlib* si procede a generare dei grafici per visualizzare come si distribuiscono le previsioni rispetto ai valori reali



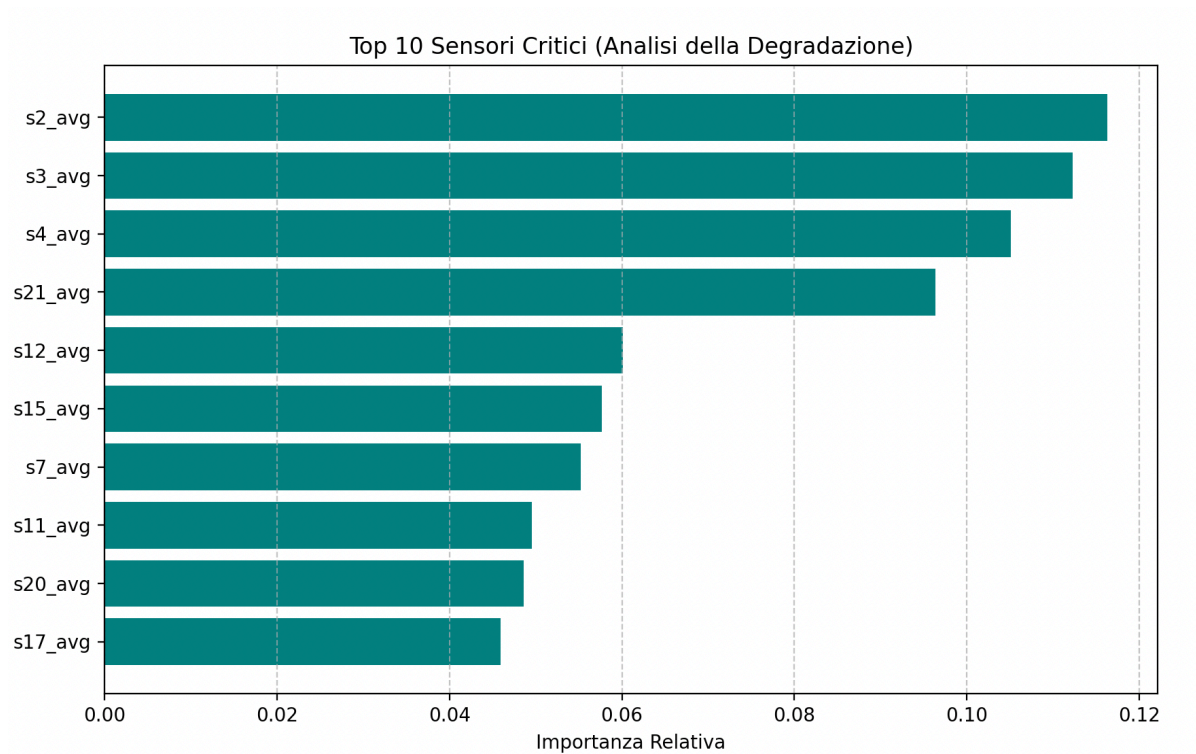
RUL predetta vs reale: come si può osservare nel primo grafico, la maggior parte dei punti si concentra in una area vicina alla diagonale (predizione perfetta) ma sono presenti anche degli outlier ovvero degli esempi il cui valore è poco rappresentato nel dataset e quindi il sistema ha difficoltà a predire con precisione.



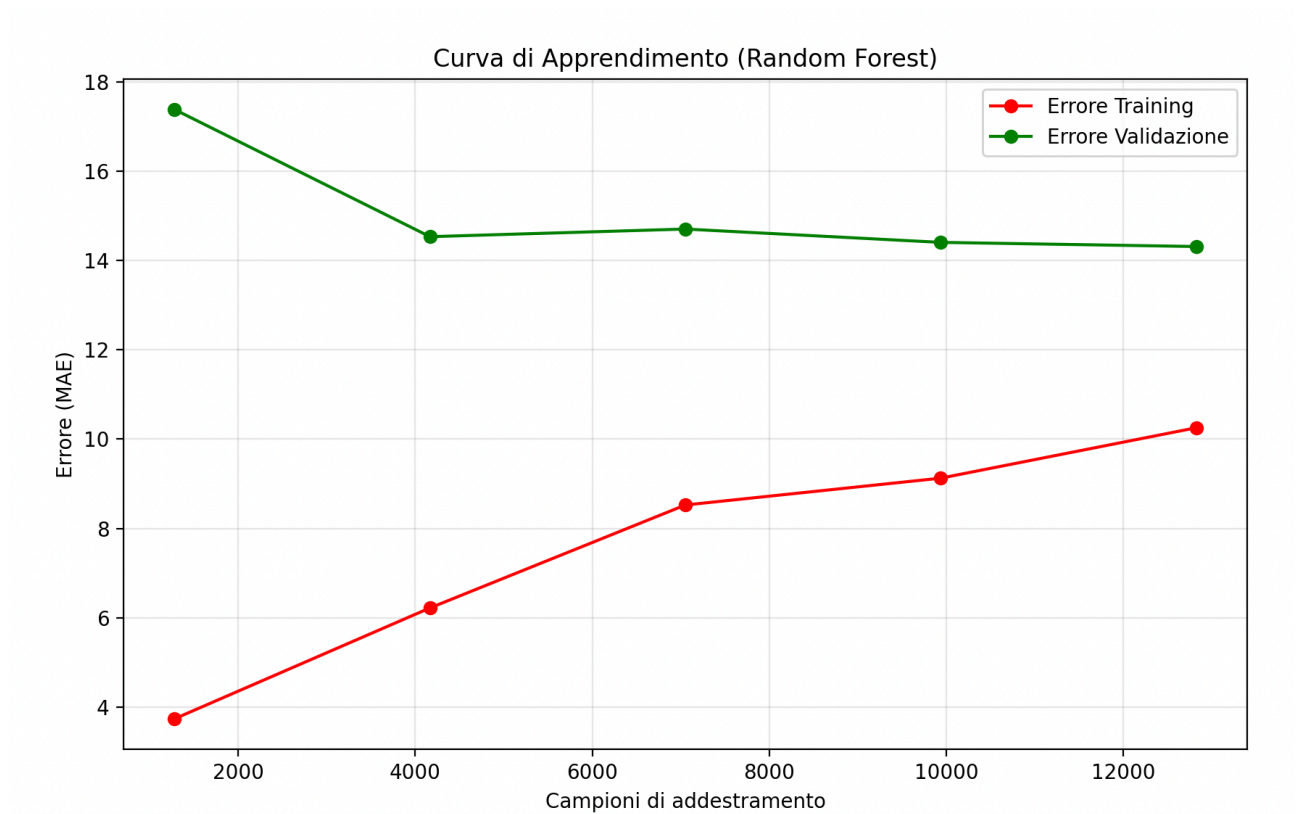
Distribuzione degli errori: in questo grafico possiamo vedere la distribuzione dell'errore definito come la differenza tra il valore reale e quello predetto.

La distribuzione è chiaramente a campana (simile a una Gaussiana) e centrata sullo zero. Questo è il segnale più importante: il modello non ha un *bias* sistematico quindi non tende né a sopravvalutare né a sottovalutare la RUL in modo costante.

Inoltre, la maggior parte degli errori si concentra nell'intervallo $[-10, +10]$ che conferma che per la stragrande maggioranza dei motori, la previsione è quasi chirurgica.



Feature Importance: questo grafico, invece, rappresenta la feature Importance che è il risultato dell'addestramento del Random Forest ed evidenzia come le medie mobili (_avg) siano spesso più determinanti dei dati grezzi per una predizione stabile. Questi dati sono molto preziosi per la realizzazione del prossimo passo di questo progetto, ovvero la realizzazione dell'ontologia e delle sue regole



Learning curve: Dopo aver discusso come si comporta il modello di regressione una volta addestrato, discutiamo di come evolve il suo errore assoluto nel tempo.

La curva verde (Validazione) scende rapidamente e si stabilizza intorno ai 6.000 campioni. Questo indica che il modello ha raggiunto il suo plateau di apprendimento: aggiungere altri dati della stessa tipologia non migliorerebbe drasticamente le performance mentre la curva rossa (Training) sale man mano che aumentano i dati.

Il fatto che le due curve tendano ad avvicinarsi (gap ridotto) dimostra che il modello generalizza bene pertanto abbiamo un buon bilanciamento tra bias e varianza e assenza di overfitting

Regressione Logistica

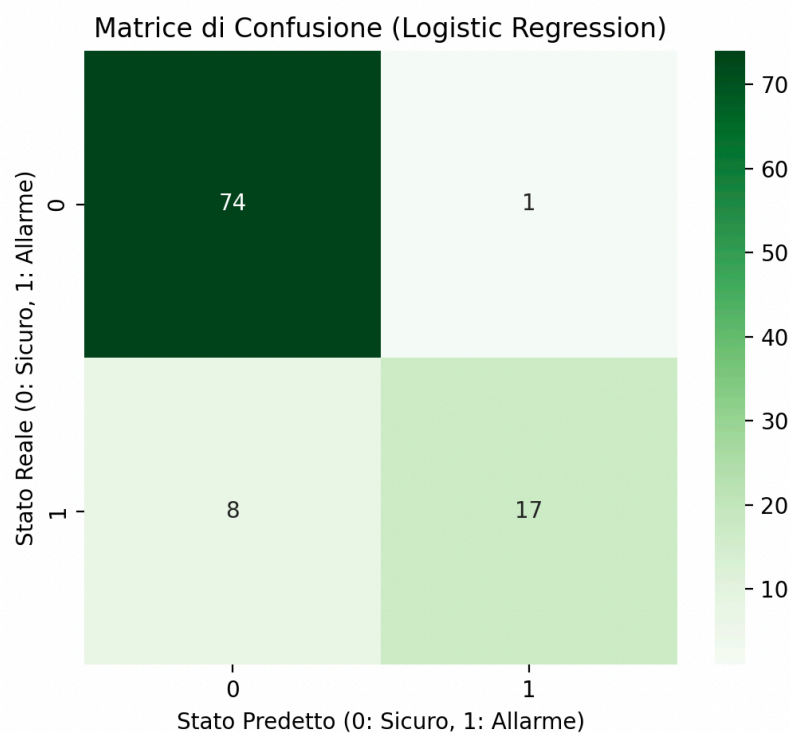
La valutazione della logistic regression serve a dimostrare che il sistema ha perfettamente compreso quando lanciare o meno l'allarme (Safe/Alarm).

Effettuando dei test sulle metriche di errore si ottengono questi risultati:

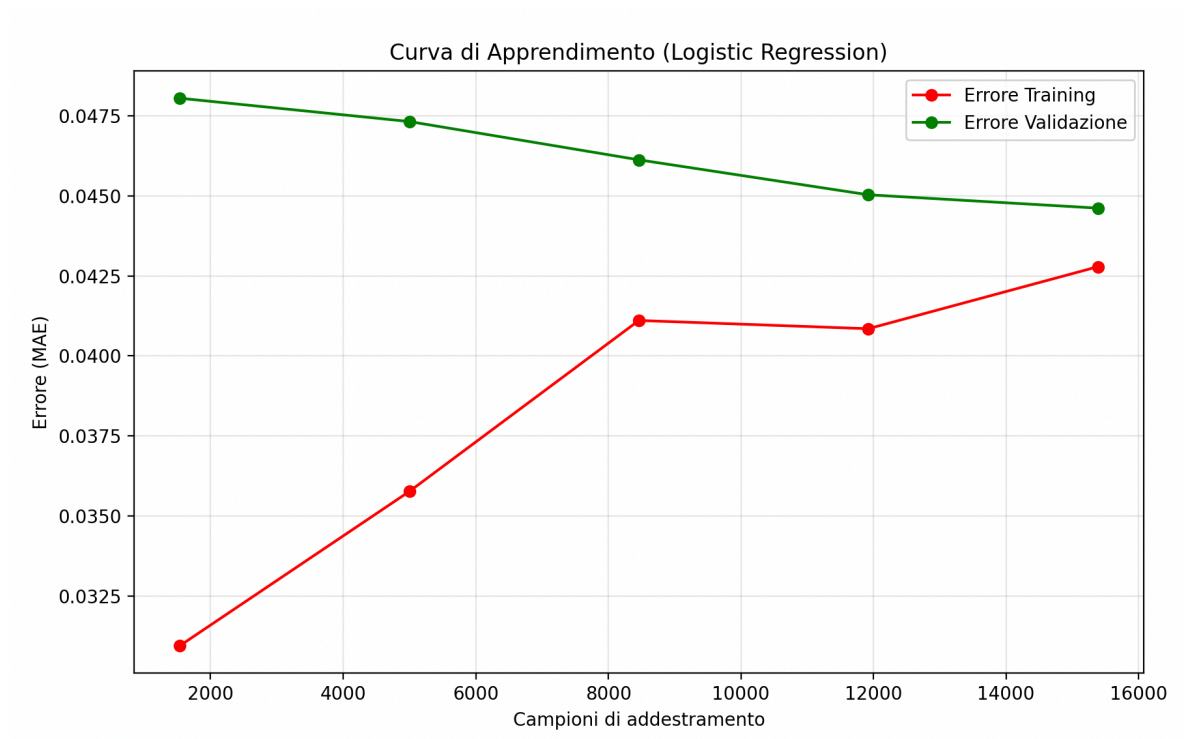
Accuracy: 95.52%

F1: 85.82%

Questi risultati indicano che il modello azzecca lo stato corretto quasi 96 volte su 100 (Accuracy). È un valore eccellente che dimostra la solidità delle feature selezionate mentre il valore della F1 è leggermente inferiore all'accuratezza in quanto nel dataset ci sono molti più motori "sani" che "in allarme", il che è normale in ambito industriale. Questo conferma che il modello è comunque molto bilanciato tra Precision (non dare falsi allarmi) e Recall (non perdere nessun guasto).

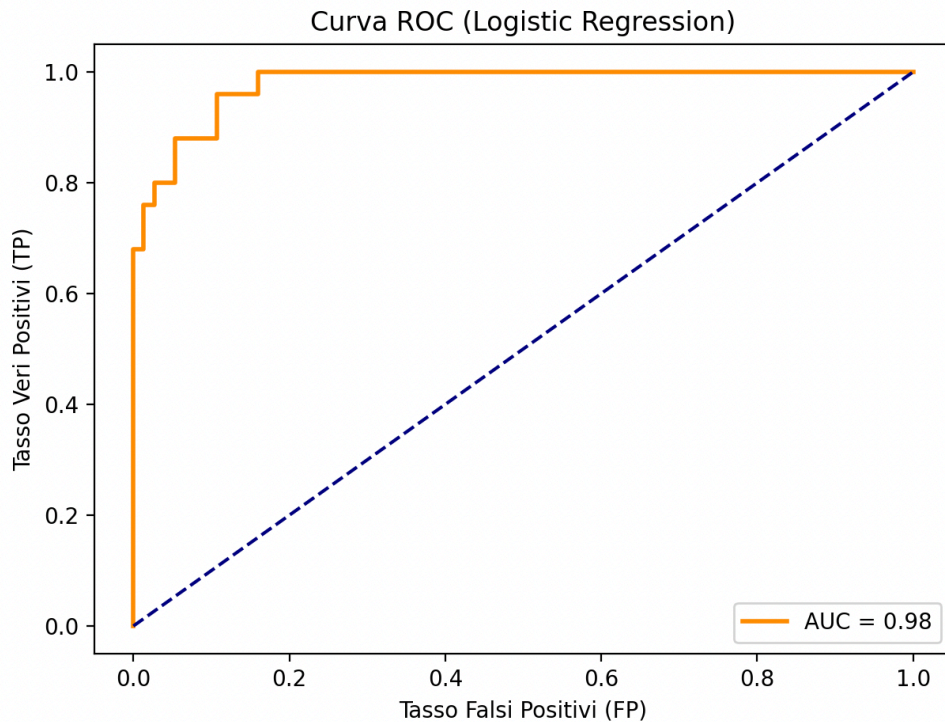


Matrice di Confusione: mostra visivamente quanti "guasti reali" sono stati ignorati (Falsi Negativi) e quanti "allarmi inutili" sono stati dati e proprio per gestire quest'ultimi abbiamo introdotto la belief network come ulteriore controllo



Learning curve: Per quanto riguarda la learning curve della logist regression, questa mostra che le curve di Training e Validation convergono molto velocemente verso il plateau. Questo indica che il modello non ha bisogno di milioni di dati per capire i segnali di guasto; le feature estratte sono molto informative.

Analizzando il grafico quindi si dimostra ancor di più l'assenza di overfitting.



Curva ROC: L'area sotto la curva (AUC) è vicina a 1.0, precisamente con un valore di 0.98 il che significa che il modello di classificazione ha una probabilità del 98% di assegnare un punteggio di rischio più alto a un motore che sta per rompersi rispetto a uno sano. Inoltre, la curva si impenna verticalmente verso l'angolo in alto a sinistra pertanto possiamo impostare una soglia di allarme molto sensibile senza aumentare drasticamente i falsi allarmi.

Rete Bayesiana

La valutazione della rete bayesiana (belief network) non si basa su metriche di valutazione standard (essendo un sistema di inferenza probabilistica e non di apprendimento) ma su scenari di inferenza e sulla capacità di correzione dei modelli di Machine Learning, in poche parole serve a dimostrare che Bayes "corregge" il Machine Learning

Per la valutazione della belief network è stata sperimentata su due casi limite

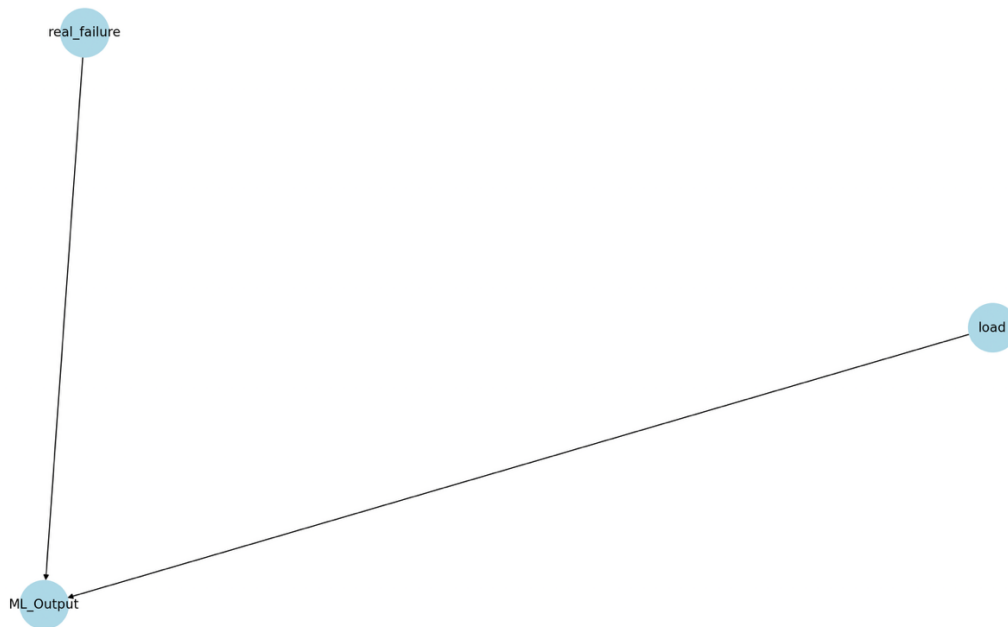
[CASO A] Carico Basso (0.2) -> Prob. Reale: 75.00%

[CASO B] Carico Alto (0.9) -> Prob. Reale: 20.67%

Questo test sui due scenari dimostra la capacità della rete di gestire l'incertezza e abbattere i falsi positivi:

- **[CASO A] Carico Basso (0.2) → Prob. Reale: 75.00%:** In questo scenario, il sensore di carico indica che il motore non è sotto sforzo estremo. Di conseguenza, l'allarme generato dal Machine Learning viene considerato altamente attendibile. La rete "conferma" il sospetto di guasto poiché non ci sono fattori esterni (carico) che giustificano anomalie nei sensori.

- [CASO B] Carico Alto (0.9) → Prob. Reale: 20.67%: Qui, nonostante il ML stia gridando "Allarme", la rete nota che il carico operativo è estremo. Consultando la propria conoscenza (CPD), la rete "sconta" l'allarme, ipotizzando che l'anomalia sia dovuta allo stress temporaneo e non a un danno strutturale.



Struttura del Grafo Bayesiano: La struttura della rete ci mostra come due cause indipendenti (*real_failure* e *load*) convergono su un unico effetto osservato (*ML_output*) quindi la rete calcola la probabilità inversa: $P(\text{real_failure} \mid \text{ML_Output}, \text{load})$

```

1. cpd_carico = TabularCPD(variable='load', variable_card=2, values=[[0.7], [0.3]])
2.
3. cpd_guasto = TabularCPD(
4.     variable='real_failure',
5.     variable_card=2,
6.     values=[[prob_guasto_false], [prob_guasto_true]]
7. )
8.
9. # Tabella di condizionamento
10. cpd_ml = TabularCPD(
11.     variable='ML_Output',
12.     variable_card=2,
13.     values=[
14.         [0.95, 0.40, 0.05, 0.01], # no failure probability
15.         [0.05, 0.60, 0.95, 0.99] # failure probability
16.     ],
17.     evidence=['load', 'real_failure'],
18.     evidence_card=[2, 2]
19. )
20. self.model.add_cpds(cpd_carico, cpd_guasto, cpd_ml)
  
```

Tabelle CPD:

- **CPD di real_failure:** mostra le probabilità a priori estratte dal Knowledge Graph. La probabilità di base del guasto è bassa (0.10 o 0.20), riflettendo il fatto che un guasto è un evento raro.
- **CPD di ML_Output:** definisce l'affidabilità del Machine Learning.
 - Si noti il valore 0.60 nella colonna dove il carico è alto ma il guasto è falso: è la probabilità che il ML dia un falso allarme a causa dello stress operativo.
 - Il valore 0.99 (Allarme ML = True | Guasto = True, Carico = Alto) indica che il sistema è quasi certo di rilevare un guasto se questo è effettivamente presente sotto sforzo

Ontologie e Ragionamento

Sommario

Il sistema si basa su una **Background Knowledge (BK)** strutturata per fornire un contesto semantico alle predizioni numeriche. Abbiamo scelto di tradurre la conoscenza del dominio in una forma computazionale ibrida:

1. **Logica Descrittiva:** Utilizzata nel Knowledge Graph per definire la struttura "statica" del mondo (chi è cosa e come è collegato).
2. **Logica Proposizionale/Clausole di Horn:** Utilizzata in Prolog per definire le regole "dinamiche" di deduzione (cosa fare se succede qualcosa)

Strumenti utilizzati

- **Protégé:** Progettazione visiva del grafo e validazione della gerarchia delle classi.
- **RDFLib:** Implementazione in Python del Knowledge Graph e gestione triple RDF.
- **SWI-Prolog:** Motore di inferenza per l'esecuzione delle regole logiche (Clausole di Horn).
- **PySwip:** Bridge di comunicazione tra il flusso dati Python e la base di conoscenza logica.

Decisioni di progetto

La progettazione della conoscenza si è articolata su tre componenti principali:

Logiche Descrittive

Abbiamo adottato i principi delle Logiche Descrittive per organizzare il mondo della fabbrica. Questo ci ha permesso di definire con precisione i Concetti (classi), i Ruoli (proprietà) e gli Individui (istanze).

Classi e sottoclassi (T-box)

Nella T-Box abbiamo definito la tassonomia. Ad esempio, abbiamo stabilito che ogni *Senior* è un *Technician* tramite la relazione di *rdfs:subClassOf*. Questo è fondamentale per l'inferenza: se una regola dice che un task richiede un Tecnico, un Senior potrà svolgerlo per ereditarietà.


```

1. # Esempio di modellazione T-Box in Knowledge_graph.py
2. self.g.add((EX.Senior, RDFS.subClassOf, EX.Technician))
3. self.g.add((EX.Junior, RDFS.subClassOf, EX.Technician))
4. self.g.add((EX.Specialist, RDFS.subClassOf, EX.Technician))
5. self.g.add((EX.Engine, RDFS.subClassOf, EX.Machine))
6. self.g.add((EX.Component, RDFS.subClassOf, EX.Part))

```

Individui (A-Box)

Nella A-Box abbiamo inserito i dati reali. Abbiamo popolato il sistema con due motori (unit_1 e unit_2) e i tecnici (Tech1, Tech2, Tech3). Ogni individuo è stato collegato ai suoi ruoli.

```

1. self.g.add((EX.unit_1, RDF.type, EX.Engine))
2. self.g.add((EX.unit_2, RDF.type, EX.Engine))
3. self.g.add((EX.Tech1, RDF.type, EX.Senior))
4. self.g.add((EX.Tech2, RDF.type, EX.Junior))
5. self.g.add((EX.Tech3, RDF.type, EX.Specialist))

```

Proprietà

Grazie alle proprietà descritte in seguito, il sistema è in grado di navigare la struttura fisica della fabbrica.

Proprietà (Ruolo)	Descrizione Logica
ex:part_of	Relazione parte-tutto tra componenti e motori.
ex:has_skill	Associazione tra tecnico e area di competenza.
ex:is_physical_part	Attributo booleano per determinare la necessità di magazzino.
ex:located_in	Associazione tra unit e area

```

1. self.g.add((EX.unit_1, EX.located_in, EX.Area_A))
2. self.g.add((EX.unit_2, EX.located_in, EX.Area_B))
3.
4. # Unit 1
5. comp1 = ["hpc", "hpt", "fan", "injector", "pressure_sensor"]
6. for c in comp1:
7.     self.g.add((EX[c], RDF.type, EX.Component))
8.     self.g.add((EX[c], EX.part_of, EX.unit_1))
9.     self.g.add((EX[c], EX.is_physical_part, Literal(True)))
10.
11. # Unit 2
12. comp2 = ["lpc", "lpt", "oil_pump", "shaft"]
13. for c in comp2:
14.     self.g.add((EX[c], RDF.type, EX.Component))
15.     self.g.add((EX[c], EX.part_of, EX.unit_2))
16.     self.g.add((EX[c], EX.is_physical_part, Literal(True)))
17.
18. # Tech 1: Senior
19. self.g.add((EX.Tech1, RDF.type, EX.Senior))
20. for area in [EX.Area_A, EX.Area_B, EX.Test_Area]:
21.     self.g.add((EX.Tech1, EX.has_skill, area))
22.
23. # Tech 2: Junior
24. self.g.add((EX.Tech2, RDF.type, EX.Junior))
25. for area in [EX.Area_A, EX.Area_B]:
26.     self.g.add((EX.Tech2, EX.has_skill, area))
27.
28. # Tech 3: Specialist
29. self.g.add((EX.Tech3, RDF.type, EX.Specialist))
30. self.g.add((EX.Tech3, EX.has_skill, EX.Test_Area))

```

Logiche proposizionale/Clausole di Horn

All'interno del file *kb_maintenance.pl* ci sono invece le regole decisionali. Sono stati configurati parametri specifici per riflettere le policy aziendali:

1. Soglia di Urgenza (urgent_maintenance):

- Parametro: $T < 24$.
- Decisione: Se il modello di Machine Learning predice un guasto entro 24 cicli operativi, l'intervento viene marcato come urgente. Questo flag influenza l'ordinamento dei tasks nel modulo CSP.

```
1. urgent_maintenance(E) :-  
2.   failure_prediction(E, T),  
3.   T < 24.
```

2. Soglia di Criticità Estrema e Seniority:

- Parametro: $RUL < 15$.
- Decisione: Indipendentemente dalla semplicità del pezzo, se la vita residua è inferiore a 15 cicli, il rischio di fermo macchina è troppo alto. Il sistema impone l'assegnazione di un tecnico Senior.

```
1. check_senior_needed(Unit, _) :-  
2.   failure_prediction(Unit, RUL),  
3.   RUL < 15.
```

3. Complessità dei Componenti (complex/1):

- Sono stati classificati come complessi i componenti core: hpc (High Pressure Compressor), hpt (High Pressure Turbine), lpt e shaft. Per questi, la regola `requires_senior(Part)` restituisce sempre vero.

```
1. requires_senior(Part) :- complex(Part).
```

4. Logica della Test Area:

- È stata introdotta la regola `goes_in_test_area`. Se un intervento coinvolge un componente complesso, il workflow post-riparazione deve obbligatoriamente includere una fase di collaudo nell'Area Test.

```
1. goes_in_test_area(E, Part) :-  
2.   part_of(Part, E),  
3.   complex(Part).
```

5. Validazione unità-componente:

- Tramite la regola `is_compatible` si verifica se una parte (componente) appartiene ad un'unità.

```
1. is_compatible(Unit, Part) :-  
2.   part_of(Part, Unit).
```

Integrazione (Modulo di Ragionamento)

Il modulo di ragionamento (Reasoning.py) funge da ponte: riceve la RUL dal Random Forest, aggiorna la KB tramite *retractall* e *assertz*, ed esegue query booleane per generare i vincoli necessari al CSP.

```
1. # AGGIORNAMENTO DATI
2. # Cancelliamo la vecchia predizione per quell'unità(motore)
3. self.prolog.retractall(f"failure_prediction({u_id}, _)")
4. # Inseriamo la nuova predizione RUL (dato che arriva dal modello ML)
5. self.prolog.assertz(f"failure_prediction({u_id}, {current_rul})")
```

La classe ReasoningModule risolve il problema della comunicazione tra il modulo probabilistico (ML) e quello deterministico (Logica). Ogni volta che il modello Random Forest produce una predizione, il modulo aggiorna la base di conoscenza Prolog.

Questa sincronizzazione permette al sistema di generare vincoli dinamici per il CSP.

```
1. # QUERY
2. is_senior = bool(list(self.prolog.query(f"check_senior_needed({u_id}, {p_id})")))
3. is_urgent = bool(list(self.prolog.query(f"urgent_maintenance({u_id})")))
4. needs_test = bool(list(self.prolog.query(f"needs_test_logic({u_id}, {p_id})")))
5. needs_warehouse = bool(list(self.prolog.query(f"requires_warehouse_stop({p_id})")))
```

Valutazione

La fase di valutazione dell'ontologia prevede la sperimentazione delle funzionalità del sistema per l'implementazione descritta fino ad ora.

CASO 1) CONTROLLO COMPONENTE-UNITA' SUPERATO

Validazione superata. Avvio analisi ML per oil_pump...

[ML] Predizione RUL per unit 2, oil_pump: 1.81 cicli

[ML SYSTEM] Risultato Integrato:

> RUL Predetta: 1.81 cicli

> Stato Classificato: ALLARME (Confidenza: 99.36%)

[REASONING] Requisiti estratti dalla KB: {'is_senior': True, 'priority': 'High', 'needs_test': True, 'needs_warehouse': True, 'target_engine': 'engine_2', 'component': 'oil_pump', 'current_rul': 1}

[KG] Contesto recuperato: Area_B

[BAYES] Probabilità reale di guasto combinata: 75.00%

CASO 2) CONTROLLO COMPONENTE-UNITA' NON SUPERATO

ERRORE DI VALIDAZIONE ONTOLOGICA:

Il componente 'oil_pump' non è presente nella unit '1'.

Il calcolo della RUL e l'analisi ML sono stati interrotti per incoerenza dati.

Come si può vedere nell'esempio l'ontologia funziona correttamente e il modulo di ragionamento che effettua il controllo di consistenza non ha segnalato inconsistenze all'interno del sistema

Ricerca di Soluzioni e Ragionamento con Vincoli

Sommario

Il sistema Smart Maintenance Hub utilizza il Ragionamento con vincoli (CSP), per implementare un insieme di variabili (Task), domini (Tecnici disponibili) e vincoli (Competenze, Livello, Capacità) in grado di mappare i requisiti astratti derivati dal modulo di ragionamento logico su risorse umane concrete, e la Ricerca di Soluzioni in spazi di stati per pianificare percorsi che minimizzano il tempo di fermo macchina.

Strumenti utilizzati

L'implementazione non sono state utilizzate librerie esterne, preferendo una codifica diretta per garantire massima trasparenza nei vincoli e l'utilizzo della libreria standard "heapq"

Decisioni di progetto

CSP (Constraint Satisfaction Problem)

Il modulo del CSP gestisce l'assegnazione dei tecnici con un algoritmo di **backtracking** seguendo una gerarchia di **vincoli** su cui prima è stato definito un dominio e delle variabili:

- **Dominio:** rappresenta l'insieme dei tecnici disponibili per eventuali tasks da portare a termine

```
1. self.technicians = [  
2.     {'id': 'Tech1', 'skills': ['Area_A', 'Area_B', 'Test_Area'], 'level': 'Senior'},  
3.     {'id': 'Tech2', 'skills': ['Area_A', 'Area_B'], 'level': 'Junior'},  
4.     {'id': 'Tech3', 'skills': ['Test_Area'], 'level': 'Specialist'},  
5.     {'id': 'Tech4', 'skills': ['Area_A', 'Area_B', 'Test_Area'], 'level': 'Senior'}  
6. ]
```

- **Variabili:** rappresenta l'insieme dei tasks da portare a termine

```
1. self.Tasks = [  
2.     {'id': 'T1', 'area': 'Area_A', 'priority': 'High'},  
3.     {'id': 'T2', 'area': 'Area_B', 'priority': 'Medium'},  
4.     {'id': 'T3', 'area': 'Test_Area', 'priority': 'High'},  
5.     {'id': 'T4', 'area': 'Area_A', 'priority': 'Low'},  
6.     {'id': 'T5', 'area': 'Area_B', 'priority': 'Low'}  
7. ]
```

- **Vincoli:** vincoli su competenze, livello e capacità per poter assegnare una specifica task ad un tecnico

```

1. # vincolo di competenza (area)
2. if task['area'] not in technician['skills']:
3.     return False
4.
5. # vincolo di carico → se il tecnico ha già 1 task assegnato non può prenderne altri
6. if technician_id in current_assignments.values():
7.     return False
8.
9. # vincolo di livello
10. if 'required_level' in task:
11.     req = task['required_level']
12.
13.     if req == 'Senior' and technician['level'] != 'Senior':
14.         return False
15.
16.     if req == 'Specialist' and technician['level'] != 'Specialist':
17.         return False
18.
19. return True

```

Inoltre, prima di avviare il backtracking, per ottimizzare, si ordinano i task per priorità (*High* > *Medium* > *Low*). Questo assicura che si cerchi di saturare le risorse migliori sui guasti più imminenti (RUL bassa).

- **Algoritmo**

- il sistema utilizza un algoritmo di Backtracking che prende una lista di tasks da assegnare e cicla su ogni task e ogni tecnico per trovare una combinazione valida basandosi sulla priorità dei task e le competenze dei tecnici. Se trova una combinazione valida, continua a cercare per il prossimo task

result = self.backtracking_search(tasks, index + 1, current_assignments)

altrimenti torna indietro (backtrack) e prova un'altra combinazione

del current_assignments[task['id']]

Ricerca di soluzioni in spazi di stati

Il modulo di ricerca di soluzioni, o meglio del percorso ha il compito di trasformare le decisioni del modulo di ragionamento in azioni concrete, calcolando il percorso ottimale che il tecnico deve compiere all'interno della fabbrica

Per la navigazione è stato implementato l'algoritmo **A* (A star)**, scelto per la sua capacità di trovare il cammino minimo (ottimalità) in tempi rapidi grazie all'uso di un'euristica:

- **Rappresentazione:** La fabbrica è modellata come un grafo pesato dove ogni nodo rappresenta un'area (Enter, Warehouse, Area_A, Area_B, Test_Area) con coordinate (x, y) specifiche.
- **Euristica:** Come funzione euristica $h(n)$ è stata adottata la **Distanza Euclidea** tra le coordinate dei nodi. Essendo la fabbrica un piano bidimensionale senza ostacoli dinamici complessi tra i nodi, la distanza in linea d'aria risulta essere un'euristica ammissibile (non sovrastima mai il costo reale) e consistente. Questo garantisce che il percorso restituito sia sempre quello a costo minimo.

Inoltre, il sistema implementa una logica di **concatenazione dei percorsi** attraverso il metodo **get_full_path**. La missione del tecnico non è sempre diretta, ma dipende dai vincoli logici identificati nella fase di *Reasoning*.

Tale metodo riceve dal main.py i flag booleani needs_warehouse e needs_test e costruisce dinamicamente una sequenza di sotto-obiettivi:

1. **Fase di Prelievo:** Se il componente richiede un ricambio fisico (needs_warehouse: True), la prima esecuzione di A* calcola il tragitto Start -> Warehouse.
2. **Fase di Intervento:** La seconda esecuzione calcola il percorso dalla posizione corrente (Start o Warehouse) verso la **Unit** di destinazione (Area_A o Area_B).
3. **Fase di Collaudo:** Se il componente è classificato come complesso (needs_test: True), viene aggiunta una tratta Unit -> Test_Area.
4. **Fase di Rientro:** Infine, il sistema pianifica il ritorno al punto di ingresso (Enter).

Valutazione

Per la valutazione e la verifica del corretto funzionamento dei moduli di CSP e Ricerca del percorso, sono state utilizzate le funzioni backtracking_search per l'assegnazione dei tasks nel rispetto dei vincoli di livello e competenza e get_full_path per la pianificazione del percorso ottimo multi-tappa, confermando la coerenza tra i requisiti logici estratti e l'effettiva esecuzione del sistema.

```
=====
CSP - ASSEGNAZIONE TECNICI
=====
ID TASK                | TECNICO   | LIVELLO
-----
Task_oil_pump          | Tech1     | Senior

=====
PIANIFICAZIONE PERCORSO OTTIMO A*
=====
Configurazione: Magazzino=True, Test Area=True
Percorso pianificato: Enter -> Warehouse -> Area_B -> Test_Area
Costo stimato (tempo): 12.00 min
=====
```

Conclusioni

Esito

Le principali componenti del progetto sono state importate all'interno del file "main.py" e sono state testate su un possibile caso d'uso date in input il motore/unit e il componente che nel nostro caso sono: unit_2 e oli_pump

```
SIMULAZIONE CASO D'USO: ANALISI PREDITTIVA E INTEGRAZIONE MODELLI
Validazione superata. Avvio analisi ML per oil_pump...

[ML] Predizione RUL per unit 2, oil_pump: 1.81 cicli

[ML SYSTEM] Risultato Integrato:
> RUL Predetta: 1.81 cicli
> Stato Classificato: ALLARME (Confidenza: 99.94%)
> Coerenza Modelli: ALTA (Entrambi i modelli concordano sullo stato)

=====
REASONING - ESTRATTO REQUISITI LOGICI
=====
[REASONING] Requisiti estratti dalla KB: {'is_senior': True, 'priority': 'High', 'needs_test': True, 'needs_warehouse': True, 'target_engine': 'engine_2', 'component': 'oil_pump', 'current_rul': 1}

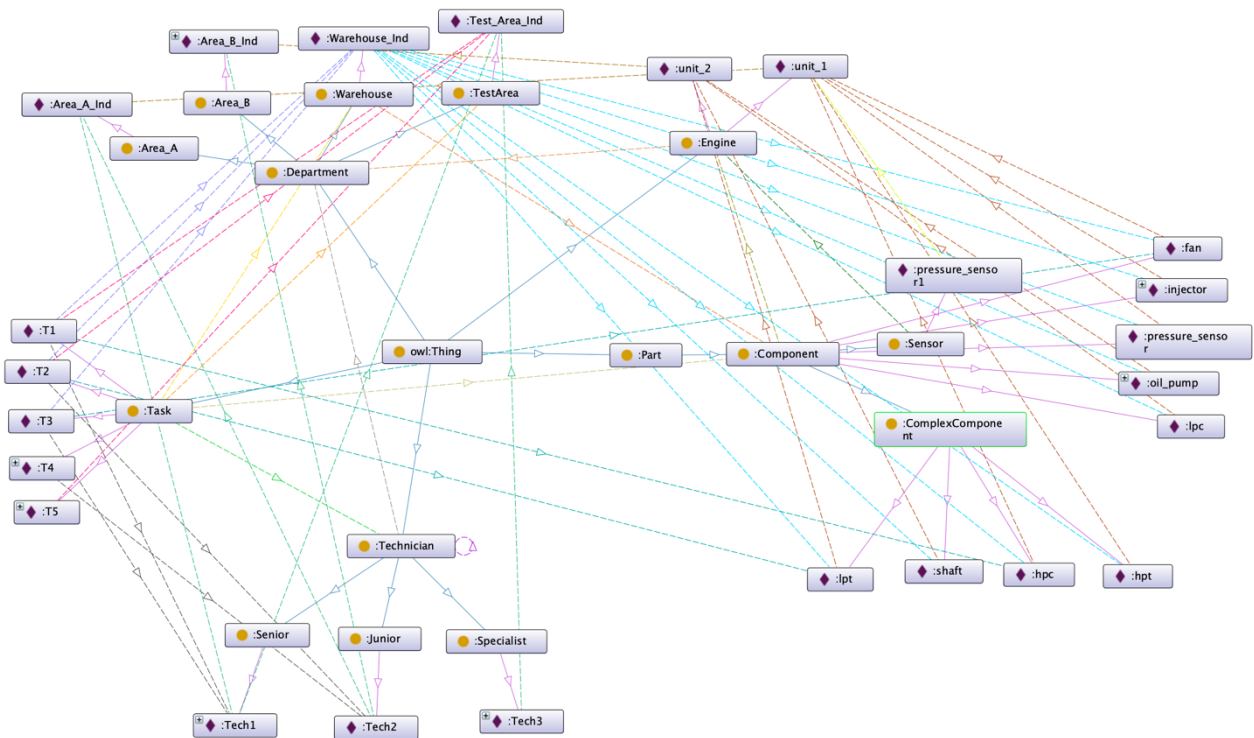
=====
KG - RECUPERO CONTEX
=====
[KG] Contesto recuperato: Area_B

=====
GESTIONE INCERTEZZA (BAYES)
=====
[BAYES] Probabilità reale di guasto combinata: 75.00%

=====
CSP - ASSEGNAZIONE TECNICI
=====
ID TASK          | TECNICO   | LIVELLO
-----
Task_oil_pump    | Tech1     | Senior

=====
PIANIFICAZIONE PERCORSO OTTIMO A*
=====
Configurazione: Magazzino=True, Test Area=True
Percorso pianificato: Enter -> Warehouse -> Area_B -> Test_Area
Costo stimato (tempo): 12.00 min
=====
```

Oltre al risultato di un possibile caso d'uso, in seguito è rappresentata l'intera struttura della knowledge graph effettuata con OWL su Protégé:



Sviluppi Futuri

Come sviluppi futuri, il sistema potrebbe integrare sensori IoT in tempo reale per aggiornare dinamicamente l'ontologia, permettendo al modulo A* di ricalcolare i percorsi in caso di ostacoli improvvisi in fabbrica. Il modulo CSP potrebbe essere esteso verso un approccio multi-agente per gestire il *ri-scheduling* dinamico dei task a fronte di imprevisti o cambi di turno dei tecnici. Inoltre, l'adozione di tecniche di AI neuro-simbolica permetterebbe una fusione più profonda tra le predizioni ML e il ragionamento logico, migliorando la gestione dell'incertezza.

Riferimenti bibliografici

- https://youtu.be/_QuGM_FW9eo?si=0BglyUMzZxLCTqgW
- <https://youtu.be/aL21Y-u0SRs?si=fu4tb3CtR4YQ7-AG>
- Libro di Testo: *Artificial Intelligence Foundations of Computational Poole, David* - Cambridge University Press
- SciKitLearn - <https://scikit-learn.org/stable/>
- Seaborn - <https://seaborn.pydata.org/>
- Matplotlib- <https://matplotlib.org/>
- Protégé - <https://protegewiki.stanford.edu/wiki/WebProtegeUsersGuide>